ALEXANDER ARRICO

Senior Software Engineer
Brunswick, GA · alex@arrico.me · (310) 740-5037
arrico.me · linkedin.com/in/aarrico · github.com/aarrico

PROFESSIONAL SUMMARY

Senior software engineer with 10+ years on production backend systems in Python, Node.js/TypeScript, and Java. Most of that work is distributed and event-driven: moving legacy monoliths onto AWS, and designing the APIs and data models underneath. Migrated 20,000+ users off a Java monolith to Node.js microservices with zero customer data loss, cut PostgreSQL query latency from seconds to tens of milliseconds, and built a serverless messaging pipeline on AWS Lambda and SQS carrying 350,000+ outbound and 12,000 inbound messages daily.

TECHNICAL SKILLS

Languages: Python, TypeScript, JavaScript, Node.js, Java, C#, SQL, Bash
Backend & APIs: FastAPI, Flask, Express.js, Next.js, REST API Design, OpenAPI/Swagger, Pydantic, asyncio
Data & Messaging: PostgreSQL, MongoDB, Elasticsearch, Redis, SQLAlchemy, Prisma, AWS SQS/SNS, Twilio, Event-Driven Architecture
Cloud & Infrastructure: AWS (Lambda, SQS, SNS, EC2, RDS, CloudWatch), GCP (App Engine, Firebase), Docker, Microservices, Serverless
DevOps & Testing: CI/CD (GitHub Actions, GitLab CI, Jenkins), DataDog, pytest, Jest, Cypress, Trunk-Based Development
Frontend: React, HTML/CSS
Practices: Agile/Scrum, Incident Management, RFC/Architecture Review, Jira, Confluence, Notion

PROFESSIONAL EXPERIENCE

Software Consultant | Freelance (Remote) *June 2024 – May 2026*

Sole engineer on a client's Discord community platform, owning architecture, deployment, and production operations end to end.

- Rewrote the client's TypeScript/Node.js application end to end on a layered domain architecture (controller, repository, domain, presentation), with PostgreSQL persistence through Prisma.
- Built scheduled session automation with downtime-recovery logic, plus an auto-discovery pattern for command and event handlers that let new features register with zero configuration.
- Owned the deployment path end to end, replacing manual deploys with multi-stage Docker builds, GitHub Actions CI/CD, and Railway. Containerized PostgreSQL kept local and production at parity.

Engineering Manager | Hourwork, Inc. (Remote) *May 2022 – November 2023*

Player-coach on a 5-person cross-functional team (3 engineers, QA, product); primary backend engineer on the platform rebuild while managing the team.

- Architected a serverless messaging platform on AWS Lambda, SQS, and SNS with Twilio that carried 350,000+ outbound and 12,000 inbound messages daily.
- Re-architected the PostgreSQL data models and cut query latency from seconds to tens of milliseconds. The resulting real-time analytics expanded the enterprise sales pipeline.
- Migrated 20,000+ users from a Java/Spring Boot monolith to Node.js microservices in 11 months with zero customer data loss and no full-service outages.
- Built out observability on DataDog and cut resolution time for critical production issues from weeks, or never, to under 24 hours.
- Moved releases from sporadic multi-month cycles to weekly with trunk-based development and automated CI/CD.
- Instituted code review standards, an RFC process, and incident management protocols; customer-reported bugs dropped 85% within two months.

Lead Software Engineer | BlueMark (Remote) *June 2021 – April 2022*

Backend lead on an e-commerce platform build, owning REST API design and the automated test strategy.

- Designed and deployed a REST API in Python/Flask on AWS EC2 with PostgreSQL, specifying the OpenAPI/Swagger contracts ahead of implementation so the frontend and backend teams could build in parallel instead of blocking on each other.
- Cut defects reaching QA and staging 70% with unit and integration coverage in pytest and Jest.
- Moved Postgres access onto SQLAlchemy, replacing hand-written queries with a single data-access layer.

Director of Technology | Optricity Corp. (Durham, NC) *January 2019 – June 2021*

Promoted to build the company's first engineering function while remaining the senior hands-on engineer on the flagship product.

- Re-architected the flagship product's XML-based project loading engine and brought load times for the largest projects from over 10 minutes to under two.
- Integrated a Python scripting engine into the core Java desktop application so customers could automate their own workflows, then used the same engine to add a regression-testing stage to CI. Customer-reported defects fell 60%.
- Built and shipped a serverless client portal on GCP (App Engine, Firebase) in under seven days, which kept client training and key business functions running through the COVID-19 shutdowns.
- Founded the company's first engineering team and grew it from 1 to 4 engineers, with hiring, onboarding, and Agile SDLC standards that raised ticket completion rates 65%.
- Migrated issue tracking to GitLab and built the company's first automated CI/CD workflows.

Senior Software Engineer | Optricity Corp. (Durham, NC) *January 2016 – January 2019*

Founding engineer on the product team, promoted to Senior, building client-facing APIs and internal tooling.

- Built a Web Services API that let clients connect their Warehouse Management Systems directly to the product and cut 10+ hours of manual data entry per client each week.
- Developed a Python/Flask quality-control scanner that validated client CSV uploads before ingest. Clients got immediate feedback and support tickets dropped 20%.
- Architected the company's secondary support application in C#/.NET with proprietary algorithms for warehouse system integration.
- Led the flagship codebase migration from SVN to Git and built the company's first Jenkins CI/CD pipeline.

PROJECTS

Starly | Distributed event processing platform github.com/aarrico/starly

Python, FastAPI, MongoDB, Elasticsearch, Redis, Docker

- Built a high-volume event ingestion pipeline: FastAPI validates into an SQS-compatible in-process queue, an async worker drains it to MongoDB as source of truth and Elasticsearch for full-text search, and Redis cache-aside serves realtime stats.
- Designed per-dependency graceful degradation: Redis outages recompute from MongoDB rather than failing requests, and storage or search outages return typed 503s naming the failed dependency class.
- Benchmarked single-node at 626 req/sec sustained ingest with 44.8 ms p99 and zero rejections; worker drain scaled near-linearly from 50 to 251 events/sec across one to four concurrent consumers.
- Traced the drain ceiling to event-loop contention. The workers share the API's loop, so drain ran 15x faster with ingest idle than in flight, and each added consumer bought drain rate at the cost of ingest p99.
- Shed 57% of writes as 503 with Retry-After once the bounded queue filled under sustained overload, with zero errors and zero data loss from the 202 responses.
- Documented the queue's delivery guarantees, its divergences from real SQS, failure modes, and 10x scaling limits in an architecture RFC. Tests split unit from integration runs with pytest markers.

EDUCATION

M.S. Materials Science & Engineering, University of Tennessee, Knoxville 2012 – 2014

B.S. Physics / B.A. Computer Science / B.A. Mathematics, Duquesne University, Pittsburgh 2008 – 2012